

HELEP — Patterns-in-Code Document

SEN3244 Software Architecture | Part H

Student: FANYI CHARLLSON FANYI

Matricule: ICTU20233841 **Date:** May 13, 2026

Lecturer: Engr. Tekoh Palma

Part A — Pre-implemented Patterns

A.1 Choreographed Saga

Intent: Coordinate the SOS workflow across services without a central orchestrator.

Where in HELEP:

File	Line	Role
services/sos-service/app/main.py	79	Starts the saga by publishing <code>sos.triggered</code>
services/sos-service/app/main.py	102	Emits the main incident event payload
services/sos-service/app/main.py	109	Publishes <code>sos.cancelled</code> for compensation
services/dispatch-service/app/main.py	62	Consumes the SOS event and begins matching
services/dispatch-service/app/main.py	101	Publishes <code>responder.confirmed</code> with <code>RELEASED</code>
services/dispatch-service/app/main.py	136	Publishes <code>responder.confirmed</code> with <code>EN_ROUTE</code>
services/notification-service/app/main.py	52	Consumes downstream events and formats notifications
services/notification-service/app/main.py	67	Publishes <code>notification.sent</code>

How it works: `sos-service` emits the first event and returns immediately. `dispatch-service` reacts, assigns or releases a responder, and publishes the next event. `notification-service` independently consumes the downstream events and sends the simulated notification. No central saga controller exists.

A.2 Publish/Subscribe via Kafka

Intent: Decouple producers from consumers while preserving ordering per incident.

Where in HELEP:

File	Line	Role
services/*/app/events.py	79	<code>publish()</code> sends messages to Kafka
services/*/app/events.py	95	<code>consume()</code> reads in a consumer group

How it works: Each service publishes to a topic and consumer groups read independently. The code uses `key=incident_id` so all events for one incident stay ordered on the same partition.

A.3 Repository

Intent: Hide SQLite access behind collection-like functions.

Where in HELEP:

File	Key functions
<code>services/user-service/app/db.py</code>	<code>init()</code> , <code>insert_user()</code> , <code>find_by_phone()</code> , <code>add_contact()</code> , <code>list_contacts()</code>
<code>services/sos-service/app/db.py</code>	<code>init()</code> , <code>insert_incident()</code> , <code>cancel()</code> , <code>get()</code>
<code>services/dispatch-service/app/db.py</code>	<code>init()</code> , <code>all_free_responders()</code> , <code>reserve_responder_for()</code> , <code>release_assignment()</code>
<code>services/notification-service/app/db.py</code>	<code>init()</code> , <code>record()</code> , <code>list_all()</code>
<code>services/analytics-service/app/db.py</code>	<code>init()</code> , <code>bump_event()</code> , <code>log_incident()</code> , <code>zone_summary()</code>

A.4 Strategy

Intent: Swap responder-matching algorithms without changing callers.

Where in HELEP:

File	Line	Role
<code>services/dispatch-service/app/matching.py</code>	27	<code>Matcher</code> protocol / base type
<code>services/dispatch-service/app/matching.py</code>	31	<code>NearestMatcher</code> concrete strategy
<code>services/dispatch-service/app/matching.py</code>	42	<code>CredibilityWeightedMatcher</code> concrete strategy
<code>services/dispatch-service/app/matching.py</code>	57	<code>matcher()</code> factory
<code>services/dispatch-service/app/matching.py</code>	58	Module-level <code>MATCHER</code> selection

How it works: The active algorithm is selected from configuration, and the caller simply asks for `pick()`. That keeps `dispatch-service` open for future algorithms like zone-priority or hybrid scoring.

A.5 Outbox-lite

Intent: Reduce the chance of losing an event between a database write and a Kafka publish.

Where in HELEP: `services/sos-service/app/main.py:79-102`

The service persists the incident before publishing the event. This is still lighter than a full durable outbox table, so it is best described as outbox-lite.

A.6 Circuit Breaker

Intent: Stop repeated retries against a failing broker and allow recovery time.

Where in HELEP:

File	Line	Role
<code>services/*/app/events.py</code>	58	<code>class CircuitBreaker</code>
<code>services/*/app/events.py</code>	65	<code>allow()</code> gate
<code>services/*/app/events.py</code>	79	<code>publish()</code> uses the breaker
<code>services/*/app/events.py</code>	95	<code>consume()</code> depends on Kafka health

The breaker is intentionally simple in the code base. The point of the pattern is still visible: prevent repeated failure loops and surface readiness problems quickly.

Part B — Patterns Added

B.1 CQRS

Intent: Separate write-side commands from read-side queries.

Where in HELEP:

File	Line	Role
<code>services/analytics-service/app/main.py</code>	40	Consumes the event stream
<code>services/analytics-service/app/main.py</code>	42	Updates event counters
<code>services/analytics-service/app/main.py</code>	45	Logs incident projections
<code>services/analytics-service/app/main.py</code>	47	Logs zone-hit projections
<code>services/analytics-service/app/main.py</code>	75	Read-only <code>/stats/zones</code> endpoint
<code>services/analytics-service/app/main.py</code>	85	Read-only <code>/stats/events</code> endpoint

`analytics-service` never handles write commands from other services. It consumes events and serves projections, which makes it a clear CQRS read model.

B.2 Sidecar-like Observability via Kubernetes Scraping

Intent: Add observability without changing application logic.

Where in HELEP:

File	Role
<code>charts/helep/charts/user-service/templates/service.yaml</code>	Prometheus scrape annotations
<code>charts/helep/charts/sos-service/templates/service.yaml</code>	Prometheus scrape annotations
<code>charts/helep/charts/dispatch-service/templates/service.yaml</code>	Prometheus scrape annotations
<code>charts/helep/charts/notification-service/templates/service.yaml</code>	Prometheus scrape annotations
<code>charts/helep/charts/analytics-service/templates/service.yaml</code>	Prometheus scrape annotations

The Services are annotated for Prometheus discovery, so metrics can be scraped without embedding monitoring code into the business logic.

Part C — Anti-pattern Avoided

Shared Database Across Services

Why avoided: Each service owns its own `db.py` and its own local SQLite file. No service reads or writes another service's database.

Evidence: `services/user-service/app/db.py`, `services/sos-service/app/db.py`, `services/dispatch-service/app/db.py`, `services/notification-service/app/db.py`, `services/analytics-service/app/db.py`

This avoids the shared-database anti-pattern and keeps service boundaries explicit.

End of Patterns Document — 8 patterns discussed, 6 pre-implemented or stubbed, 2 added, 1 anti-pattern avoided